Prêt à dépenser
Smart | Fast | Secure

Prêt à dépenser
Architecture de Scoring Binaire pour les Prêt à la Consommation

PARTIE 2 / 2
MLOps. Déploiement & Monitoring du Modèle de Scoring Crédit

Rafael CEREZO MARTIN

April 2026

Prêt à dépenser
Smart | Fast | Secure

Introducción

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

Objectifs Partie 1/2

Département demandeur : Direction des Risques Financiers

Problématique : Inclusion financière des clients sans historique de crédit tout en minimisant le risque de défaut.

Objectif principal : Développer un modèle de classification automatisé pour prédire la probabilité de faillite et industrialiser son cycle de vie via une approche MLOps de bout en bout.

Standard Industriel (MLOps)

- **Tracking avec MLflow :** Suivi rigoureux des expérimentations, des hyperparamètres et des métriques métiers pour garantir la reproductibilité totale.
- **Model Registry & Serving** : Stockage centralisé et versionnage des modèles pour faciliter le passage de l'expérimentation au test de serving.
- **Optimisation Avancée :** Utilisation d'Optuna ou GridSearchCV pour maximiser les performances techniques et métiers.

Prêt à dépenser
Smart | Fast | Secure

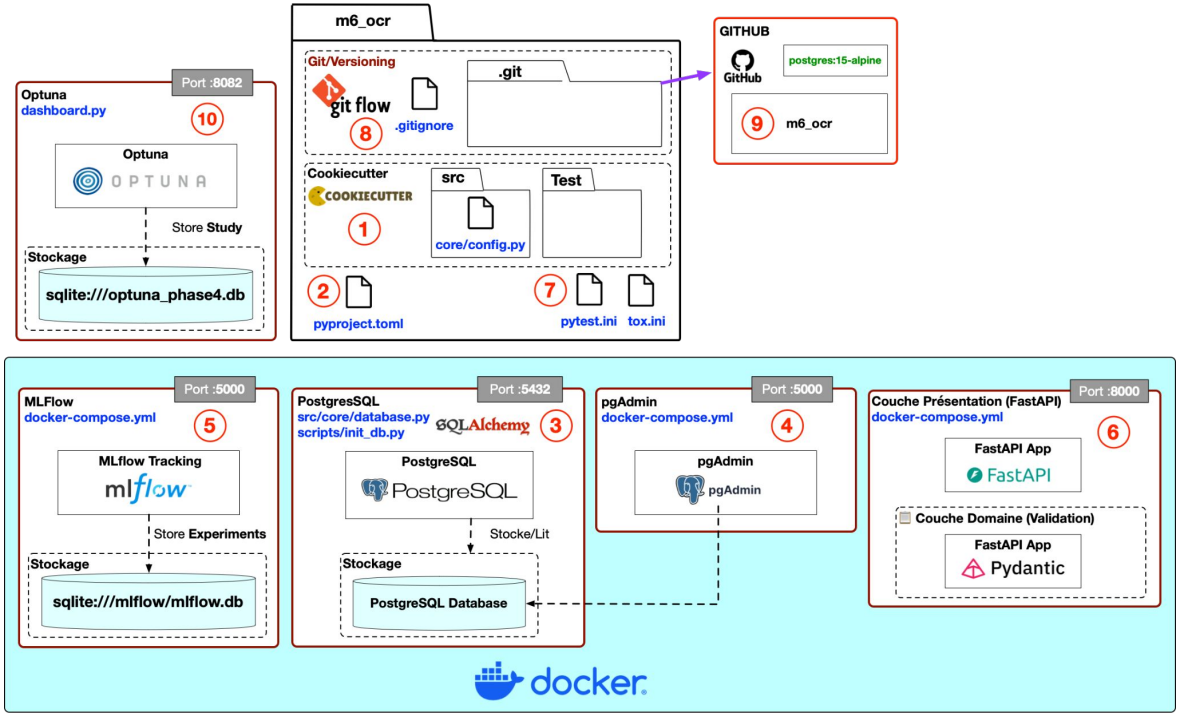
1

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

Architecture de Scoring Binaire pour les Prêt à la Consommation Part 1/2



Analyse Exploratoire (Nettoyage et Préparation)

Prêt à dépenser
Smart | Fast | Secure

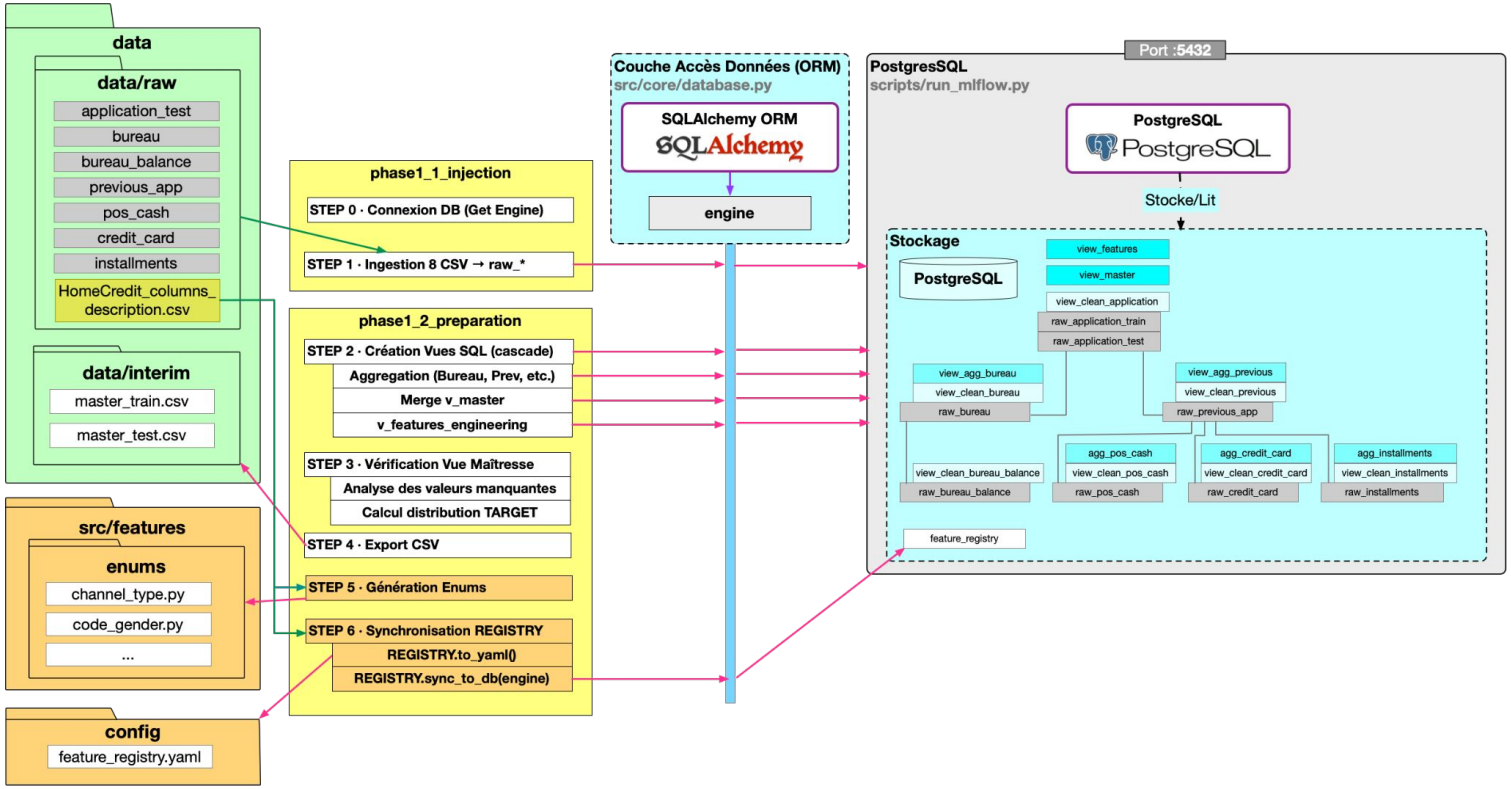
1

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

STEP 1. Pipeline



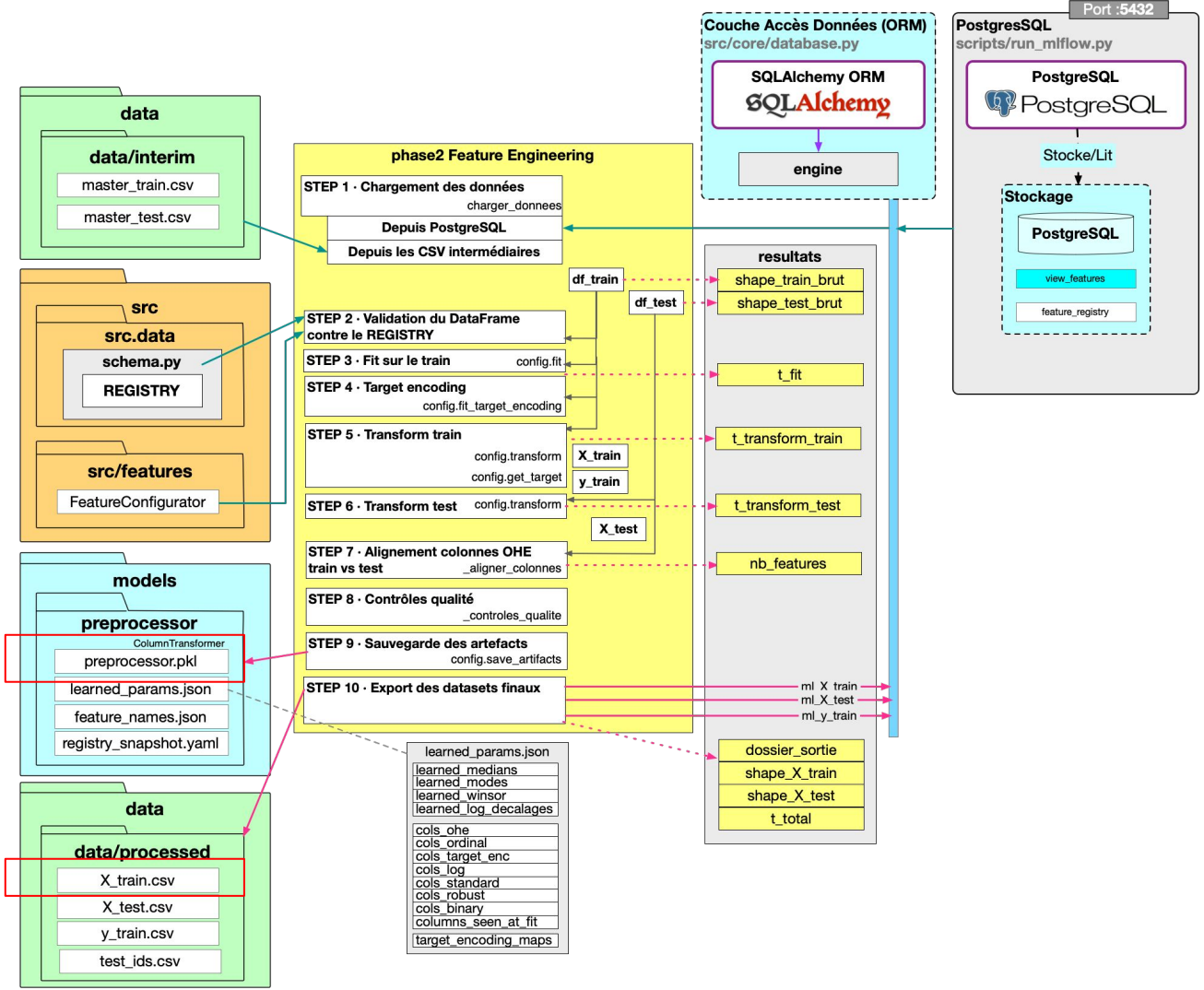
Prêt à dépenser
Smart | Fast | Secure

1

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan



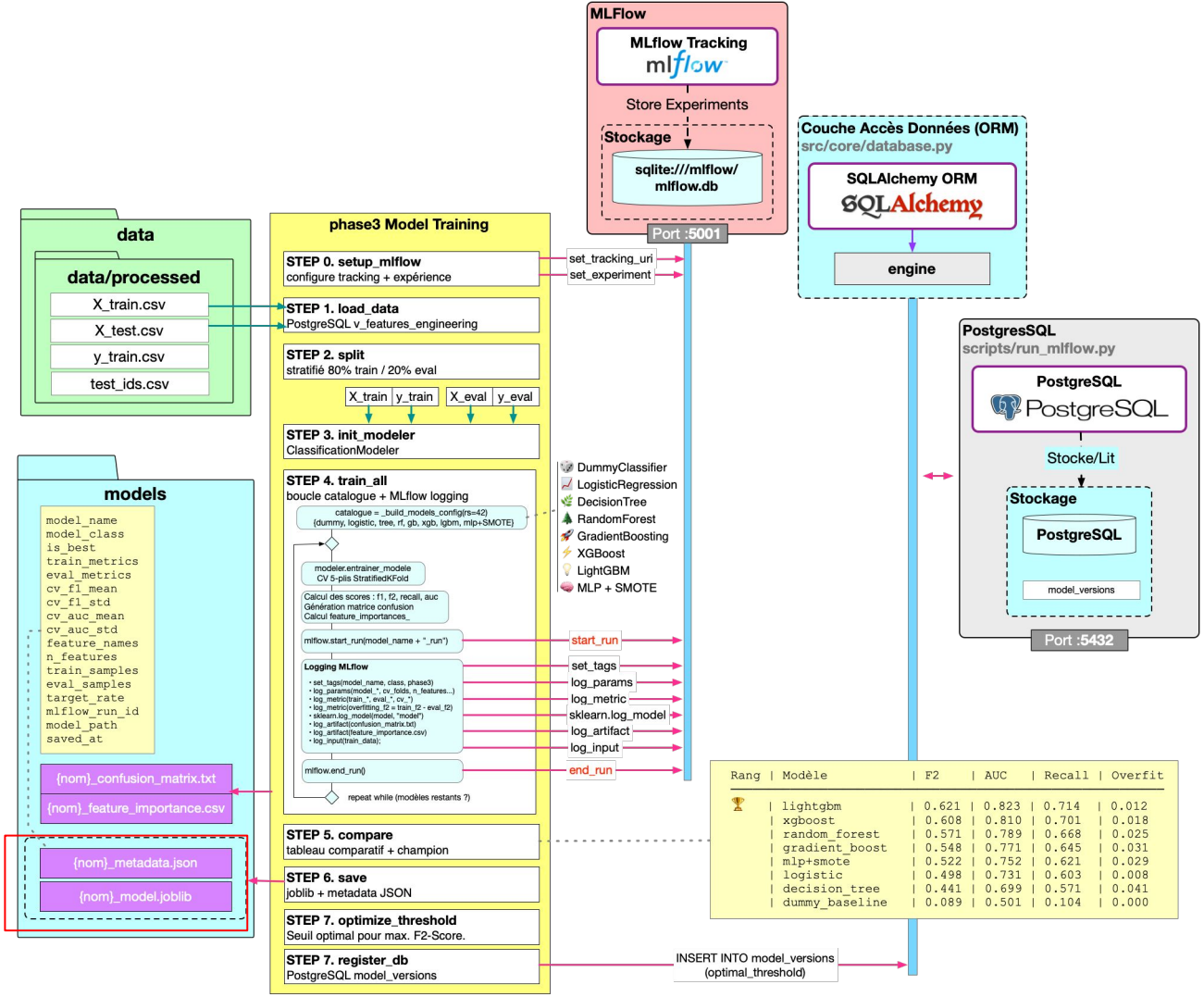
Prêt à dépenser
Smart | Fast | Secure

1

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan



Etape finale apres optimisation: Interprétabilité du modèle

Comprendre les décisions de l'algorithme (éviter l'effet "boîte noire")

Prêt à dépenser
Smart | Fast | Secure

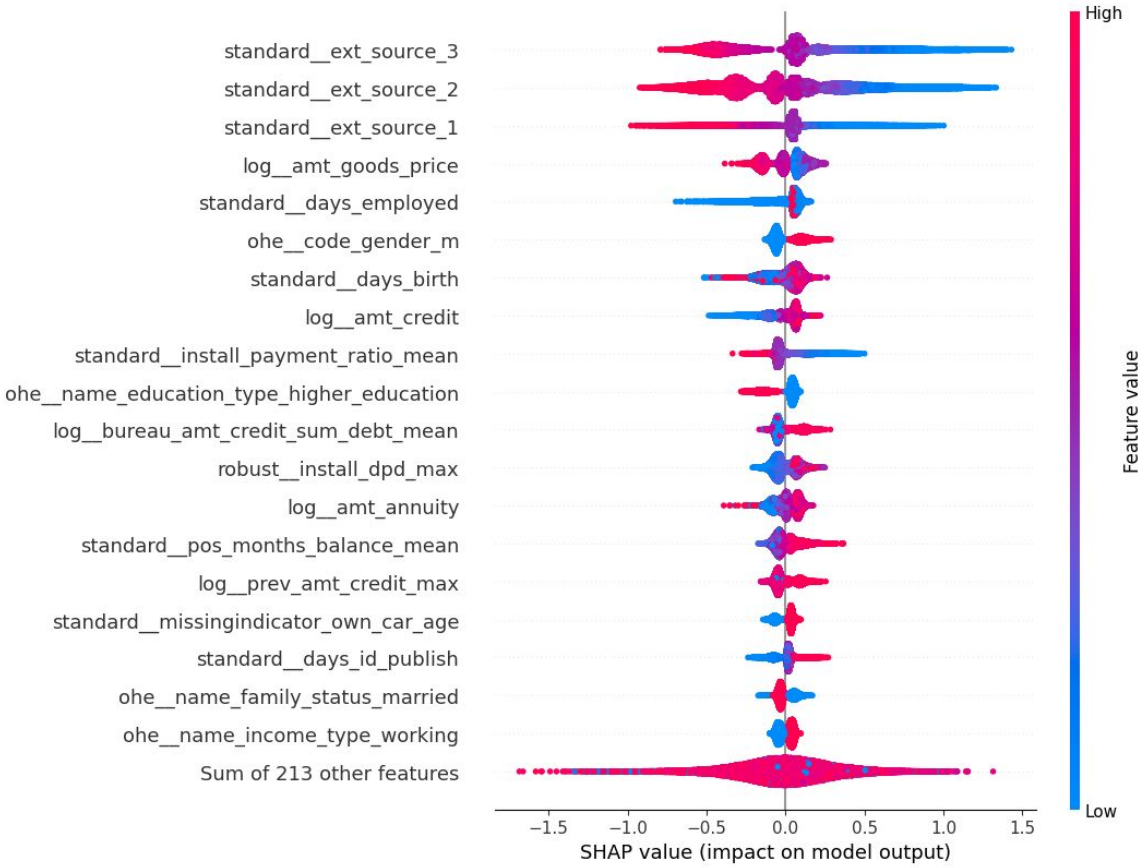
1

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

📊 **Beeswarm Plot — Top 20 variables (impact sur la probabilité de défaut)**



Prêt à dépenser
Smart | Fast | Secure

2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

Objectifs Partie 2/2 Déploiement & Monitoring Proactif

Industrialisation de l'API : Développement d'une interface robuste (FastAPI/Gradio) pour une consommation des scores en temps quasi-réel par le département "Crédit Express".

Optimisation de la Performance : Profilage du code et utilisation de formats comme **ONNX** pour minimiser le temps d'inférence et optimiser l'utilisation des ressources.

Interface utilisateur (user-friendly) interactive permettant de saisir et d'envoyer les données client à l'API pour obtenir un score de crédit et ses explications SHAP.

Monitoring & Observabilité : Mise en place d'un **dashboard** (Streamlit/Evidently AI) pour le suivi de la **latence**, de la distribution des scores et la détection précoce du **Data Drift**.

Conteneurisation & Agilité : Standardisation de l'environnement via **Docker** pour garantir la portabilité et la cohérence entre le développement et la production.

Pipeline CI/CD Automatisé : Automatisation du cycle de déploiement (GitHub Actions) incluant des tests unitaires rigoureux (Pytest) pour valider l'intégrité du modèle à chaque mise à jour.

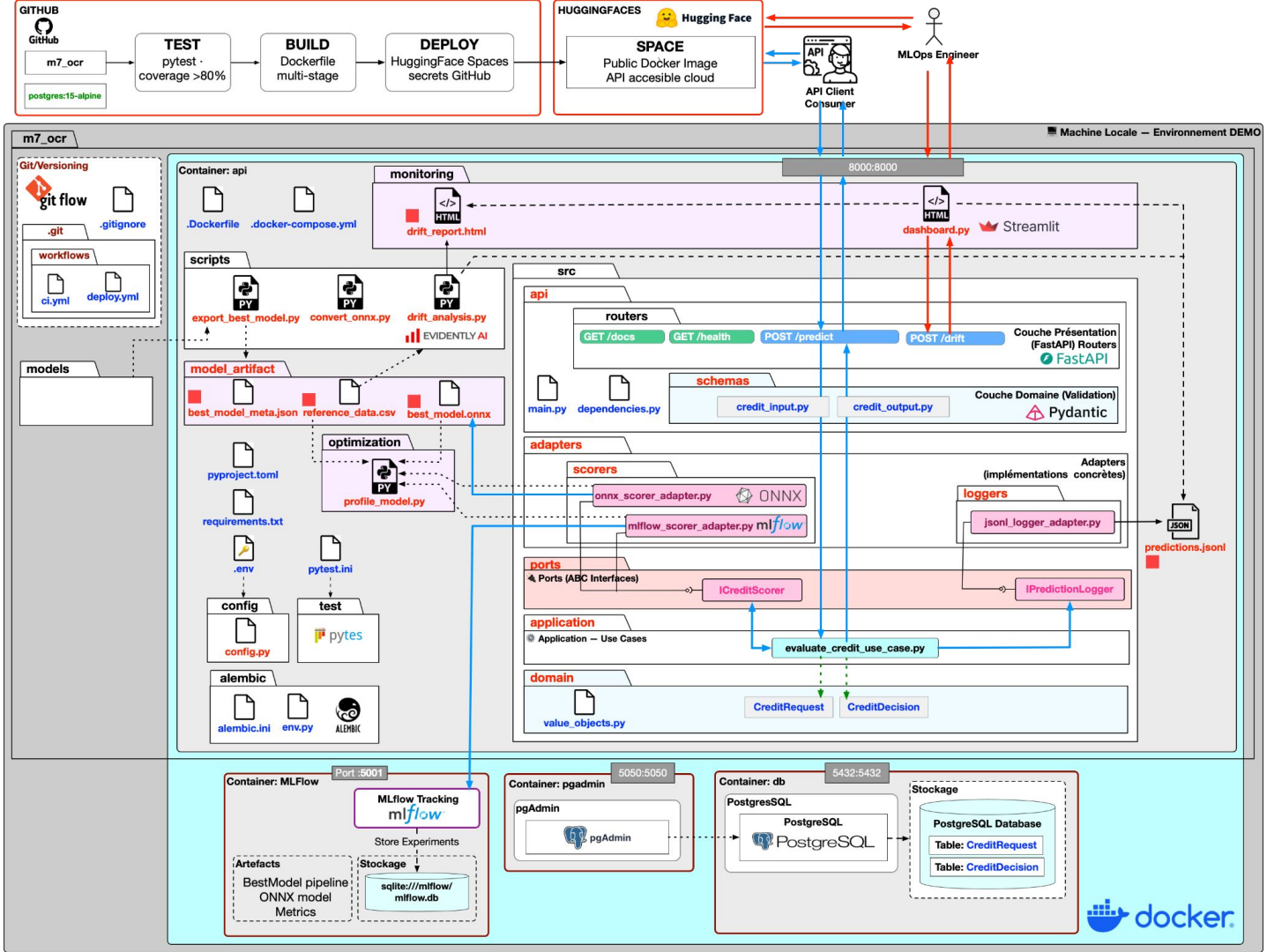
Prêt à dépenser
Smart | Fast | Secure

2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan



Prêt à dépenser
Smart | Fast | Secure

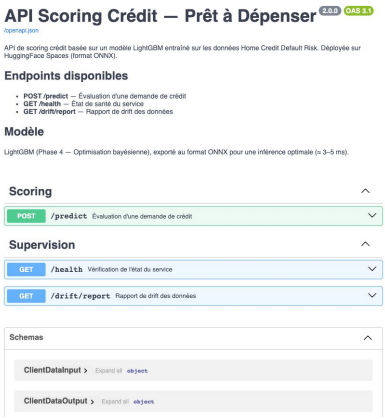
2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

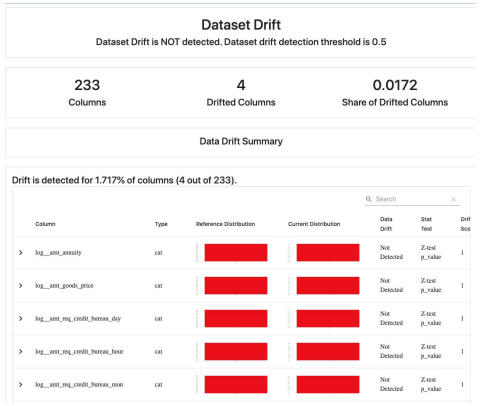
Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

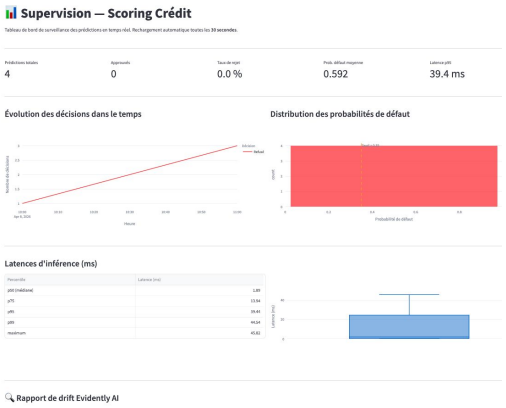
Docker Service: API
Port 8001



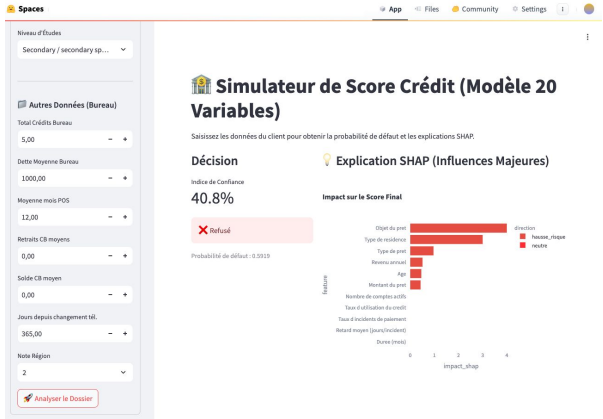
Rapport Evidently AI
drift_report.html



Docker Service: Dashboard
Port 8501



Docker Service: Simulator
Port 8502



Script. Export Best Model

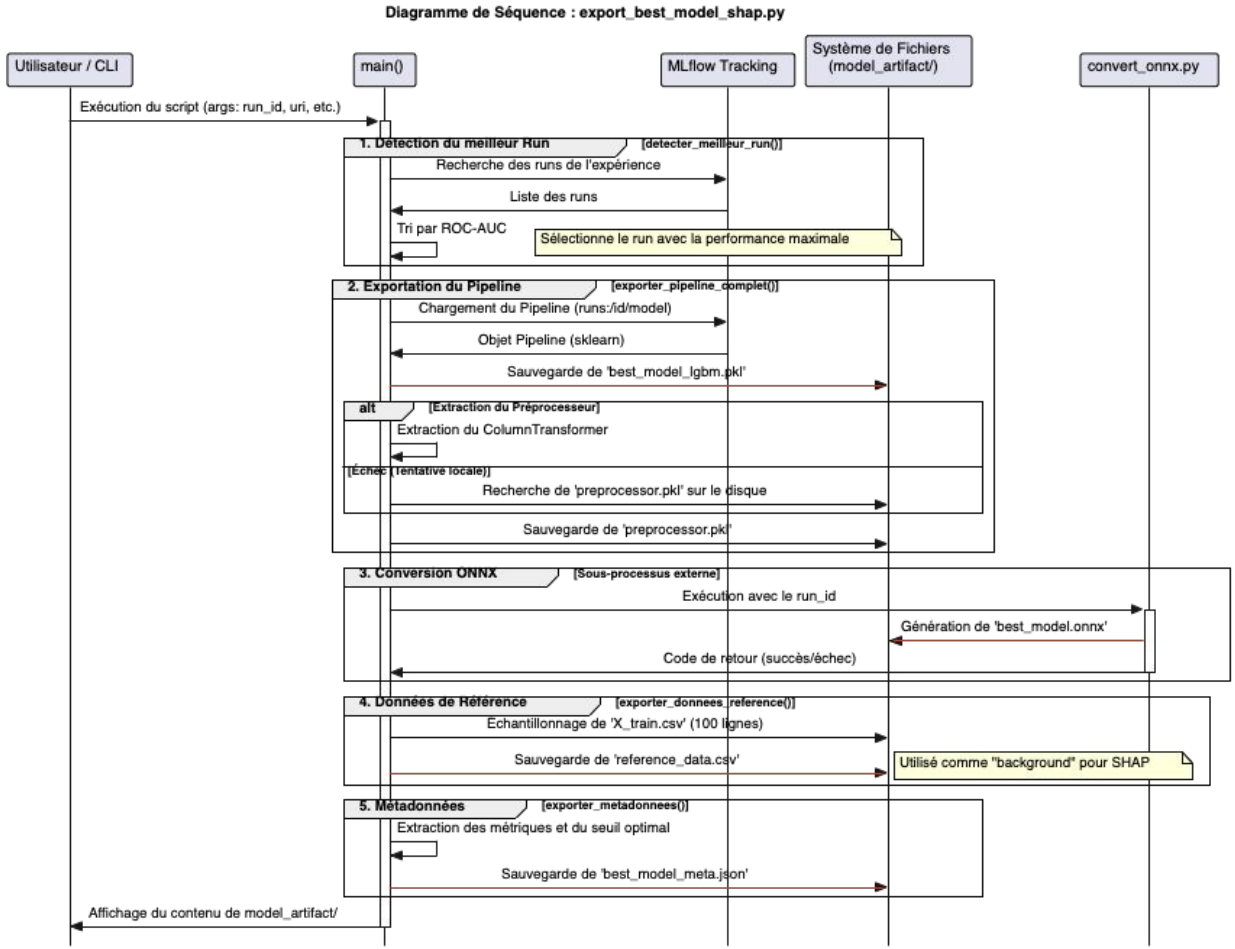
Prêt à dépenser
Smart | Fast | Secure

2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan



best_model.pkl

preprocessor.pkl

best_model.onnx

reference_data.csv

best_model_meta.json

Use Case: Evaluate Credit (Calcul et Traitement des SHAP Values)

Étape 3 - Implémentez le stockage et l'analyse des données de production

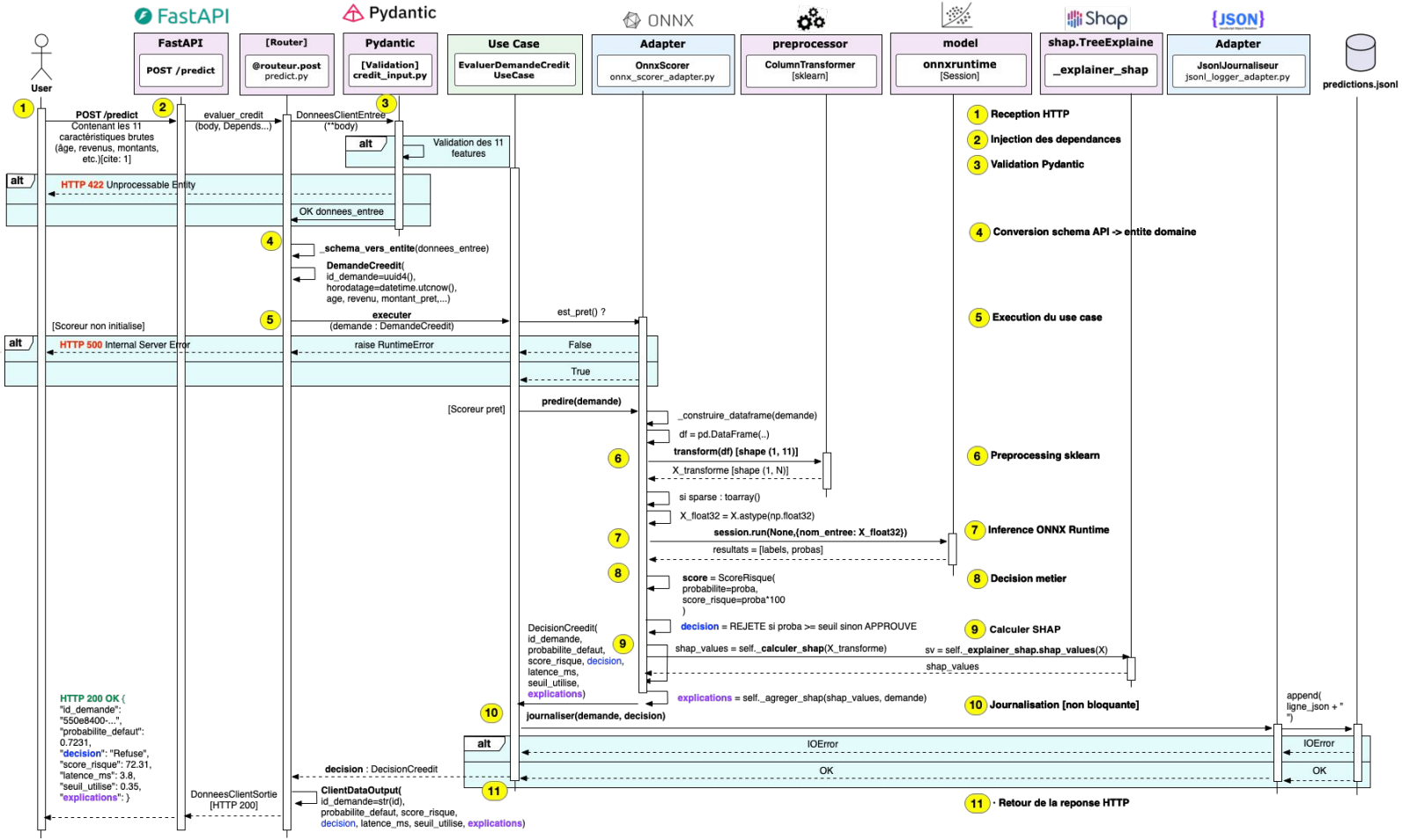
Prêt à dépenser
Smart | Fast | Secure

2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan



Use Case: Evaluate Credit (Calcul et Traitement des SHAP Values)

Étape 3 - Implémentez le stockage et l'analyse des données de production

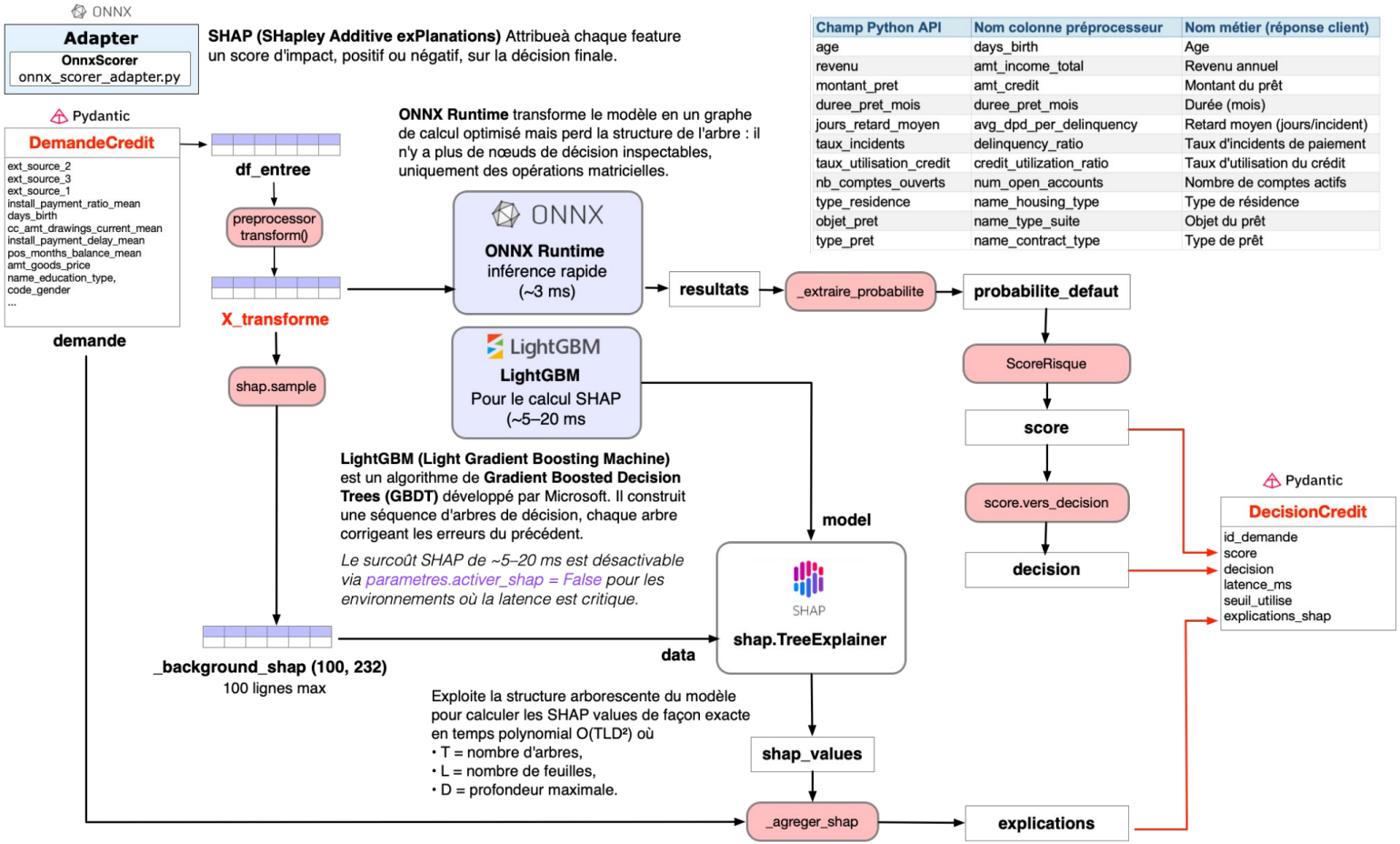
Prêt à dépenser
Smart | Fast | Secure

2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan



Champ Python API	Nom colonne préprocesseur	Nom métier (réponse client)
age	days_birth	Age
revenu	amt_income_total	Revenu annuel
montant_pret	amt_credit	Montant du prêt
duree_pret_mois	duree_pret_mois	Durée (mois)
jours_retard_moyen	avg_dpd_per_delinquency	Retard moyen (jours/incident)
taux_incidents	delinquency_ratio	Taux d'incidents de paiement
taux_utilisation_credit	credit_utilization_ratio	Taux d'utilisation du crédit
nb_comptes_ouverts	num_open_accounts	Nombre de comptes actifs
type_residence	name_housing_type	Type de résidence
objet_pret	name_type_suite	Objet du prêt
type_pret	name_contract_type	Type de prêt

Script. Analyse du Drift du Modele

Étape 3 - Implémentez le stockage et l'analyse des données de production

Prêt à dépenser
Smart | Fast | Secure

2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

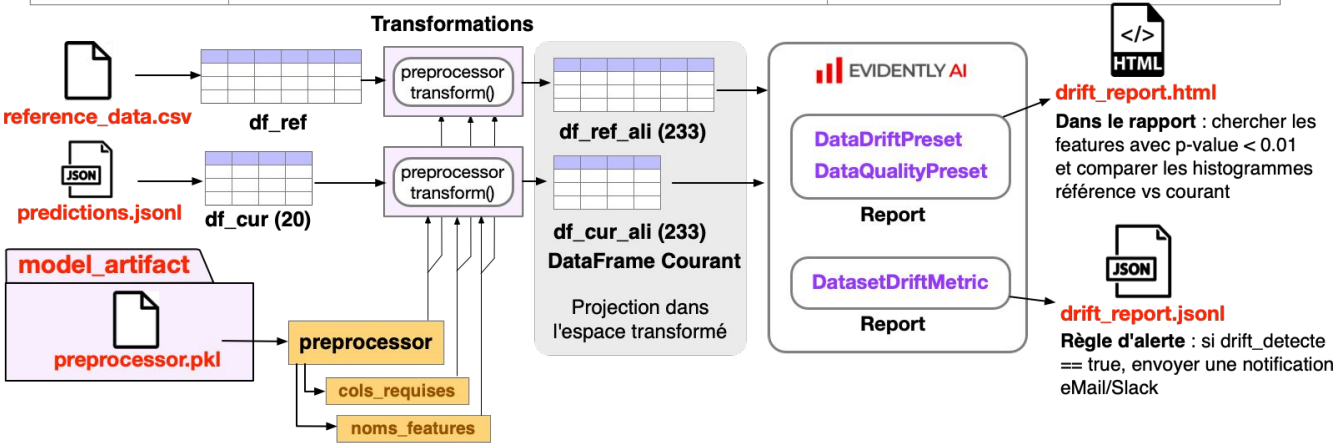
Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

PY
drift_analysis.py

Les données entrantes en production ressemblent-elles encore aux données d'entraînement ?
Si la distribution des features change (drift), le modèle produit des prédictions moins fiables, même sans aucune modification du code.

Métrique Evidently	Rôle	Sensible à
DataDriftPreset	Analyse le drift colonne par colonne. Utilise le test statistique adapté à chaque type (KS pour numérique, Chi² pour catégoriel)	Changements de distribution sur les 232 features
DataQualityPreset	Analyse la qualité des données : valeurs manquantes, doublons, valeurs hors plage	Valeurs NaN, constantes, plages anormales
DatasetDriftMetric	Résumé global : nombre de features driftées, score de drift global	Vue agrégée pour le mode --format json



Type de feature	Test utilisé	H0 (hypothèse nulle)	Drift si
Numérique continue	Kolmogorov-Smirnov (KS)	Les deux distributions sont identiques	p-value < 0.05
Catégorielle (OHE binaire)	Chi-carré (χ^2)	Les fréquences des catégories sont identiques	p-value < 0.05
Catégorielle (string)	Chi-carré (χ^2)	Même distribution des modalités	p-value < 0.05

Seuil de drift global (drift_share): par défaut, Evidently considère que le dataset dérive si plus de 50 % des features ont un drift individuel.

Script. Profilage de performance du modèle

Étape 4 - Analysez et optimisez les performances du modèle

Prêt à dépenser
Smart | Fast | Secure

2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

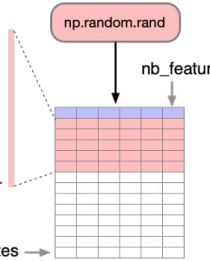
Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

profile_model.py

Le modèle ONNX est-il assez rapide pour la production ?
Il mesure la latéence d'inférence requête par requête et identifie les fonctions Python les plus coûteuses dans le pipeline d'exécution

Phase de warm-up
5 requêtes de warm-up sont effectuées et non comptabilisées dans les statistiques.
Elles permettent à ONNX Runtime d'initialiser ses buffers internes (JIT, allocation mémoire) pour que les mesures reflètent le régime permanent, pas le démarrage à froid.

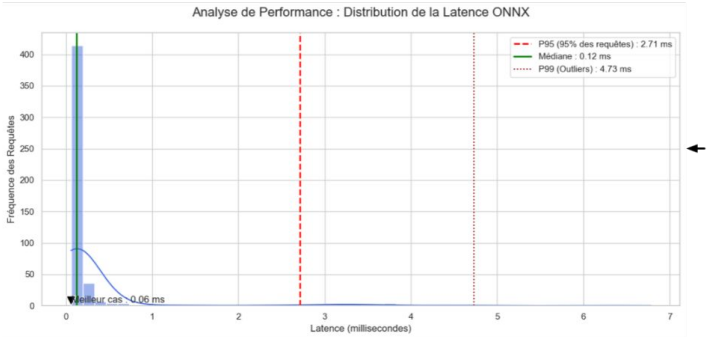


model_artifact
best_model.onnx
best_model.plk

moteur_type → **Benchmark de latéences** (mesures statistiques p50/p95/p99) → **metriques**

moteur_type
onnx - - - - - - - → model.predict_proba(x)
sklearn - - - - - → session.run(None, {input_name: x})

moteur_type → **Profilage cProfile** (inspection fonction par fonction du temps CPU consommé) → **rapport_profil**



Interprétation des percentiles

Métrique	Ce qu'elle mesure	Seuil acceptable (scoring crédit)
p50 (médiane)	Temps de réponse pour 50 % des requêtes	< 5 ms avec ONNX
p95	Temps de réponse pour 95 % des requêtes	< 10 ms (SLA typique)
p99	Pire cas hors outliers extrêmes	< 20 ms acceptable
max	Outlier absolu — peut être GC ou OS interrupt	Ignorer si isolé

Sortie console — rapport de benchmark

```
============================================================
RAPPORT DE BENCHMARK — INFÉRENCE ONNX RUNTIME
============================================================
  Moteur d'inférence......: ONNX Runtime
  Nombre de requêtes......: 500
------------------------------------------------------------
  Latence minimum.........: 2.841 ms
  Latence p50 (médiane)...: 3.127 ms      <- réponse typique
  Latence p75.............: 3.298 ms
  Latence p95.............: 4.201 ms      <- pic de charge
  Latence p99.............: 5.847 ms      <- pire cas habituel
  Latence maximum.........: 12.301 ms     <- outlier
  Latence moyenne.........: 3.219 ms
============================================================
```

Sortie JSON — format machine (--format json)

```
// optimization/rapports/benchmark_onnx.json
{
  "moteur"      : "ONNX Runtime",
  "nb_requetes" : 500,
  "latence_min" : 2.841,
  "latence_p50" : 3.127,
  "latence_p75" : 3.298,
  "latence_p95" : 4.201,
  "latence_p99" : 5.847,
  "latence_max" : 12.301,
  "latence_moy" : 3.219
}
// Note : latences_raw (liste des 500 valeurs) est omis du JSON
```

Sortie CSV — série temporelle (--format csv)

```
# optimization/rapports/benchmark_onnx_latences.csv
requete_num,latence_ms
0,3.241
1,3.108
2,2.956
...
499,3.187
```

Rapport cProfile — texte (toujours généré)

```
# optimization/rapports/cprofile_onnx.txt
         500 function calls in 1.614 seconds

   Ordered by: cumulative time
   List reduced from 247 to 20 due to restriction <20>

   ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      500    0.003    0.000    1.614    0.003  profile_model.py:168(run)
      500    0.001    0.000    1.611    0.003  backend.py:42(run)
      500    0.000    0.000    1.608    0.003  {built-in method run}
        1    0.000    0.000    0.004    0.004  numpy/random.py:221(rand)
```

Script. Profilage de performance du modèle

Étape 4 - Analysez et optimisez les performances du modèle

Prêt à dépenser
Smart | Fast | Secure

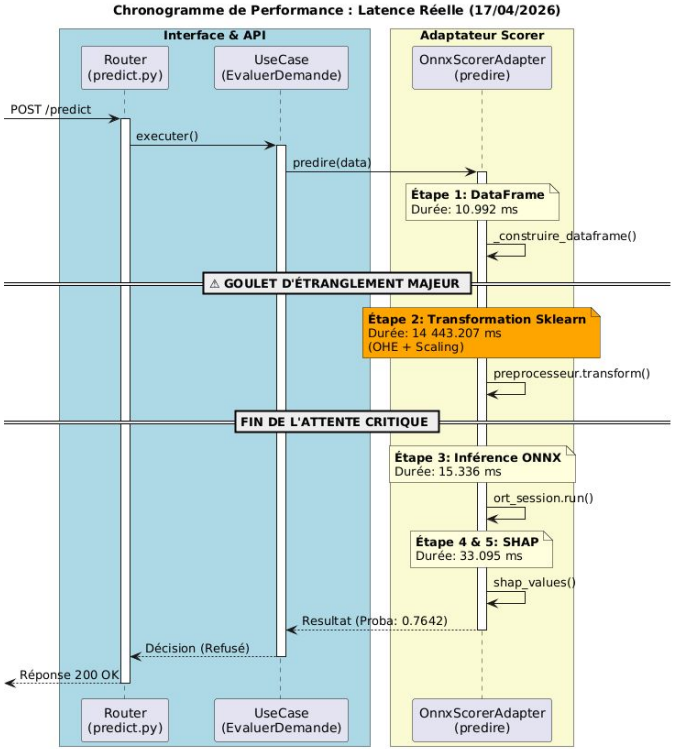
2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

API /predict Endpoint



Étape	Temps (ms)	% du total
DataFrame	11	0.08 %
Préproc	**14 443**	**99.59 %**
ONNX	15	0.10 %
SHAP	33	0.23 %
TOTAL	**14 502**	**100 %**

Script (Test Data, Inference Time)

```
============================================================
RAPPORT DE BENCHMARK — INFÉRENCE ONNX RUNTIME (DESPUÉS)
============================================================
  Moteur d'inférence......: ONNX Runtime
  Nombre de requêtes......: 500
------------------------------------------------------------
  Latence minimum.........: 0.049 ms
  Latence p50 (médiane)...: 0.063 ms      <- réponse typique
  Latence p75.............: 0.070 ms
  Latence p95.............: 0.107 ms      <- pic de charge
  Latence p99.............: 0.289 ms      <- pire cas habituel
  Latence maximum.........: 0.454 ms      <- outlier
  Latence moyenne.........: 0.071 ms
============================================================
2026-04-17 12:14:00,014 | INFO     | CSV exportado:
C:\Users\Public\IAE_DELL\pra_dell\m6_ocr\optimization\rapports\benchmark_onnx_latences.csv
2026-04-17 12:14:00,017 | INFO     | cProfile exportado:
C:\Users\Public\IAE_DELL\pra_dell\m6_ocr\optimization\rapports\profiling_onnx.txt

---------------------------RESUMEN FINAL----------------------------
   Scikit-Learn (Antes)           | p50:  1.916 ms  p95:  3.149 ms
   ONNX Runtime (Después)         | p50:  0.063 ms  p95:  0.107 ms
--------------------------------------------------------------------
AMÉLIORATION DE LA VITESSE:   x30.51 fois plus rapide avec  ONNX
====================================================================
```

Dashboard (Real Data, Response Time)

Latences d'inférence (ms)

Percentile	Latence (ms)
p50 (médiane)	2.12
p75	13.87
p95	64.1
p99	80.53
maximum	84.04

Prêt à dépenser
Smart | Fast | Secure

2

PARTIE 1/2 (Révision)
1.1 Architecture
1.2 Analyse Exploratoire
1.3 Feature Engineering
1.4 Expérimentations
1.5 Optimisation

Partie 2/2
2.1 Architecture
2.2 Best Model
2.3 Use Case
2.4 Drift Analyse
2.5 Performance

Bilan

Bilan du projet réalisé

Concevoir un système de suivi du cycle de vie du modèle d'apprentissage.

Industrialisation complète

Du modèle MLflow versionné à une API Docker-ready déployée sur HuggingFace en pipeline CI/CD automatisé

Robustesse & tests

Validation stricte des inputs, 14 tests unitaires couvrant cas limites et erreurs, couverture > 80%

Monitoring proactif

Détection de drift (Evidently AI), dashboard Streamlit, latence p95/p99 — Pas de boîte noire en production (SHAP Values)

Optimisation mesurée

cProfile identifie les goulots, ONNX Runtime réduit la latence 2-5× — optimisation justifiée par données réelles

Prêt à dépenser
Smart | Fast | Secure

Q & A

Rafael CEREZO MARTIN April 2026